Dijkstra's Algorithm – Code & Execution Output

1. Source Code (C)

```c
// Dijkstra's Algorithm in C
#include <stdio.h>
#define INFINITY 9999
#define MAX 10

void Dijkstra(int Graph[MAX][MAX], int n, int start);

void Dijkstra(int Graph[MAX][MAX], int n, int start) {
    int cost[MAX][MAX], distance[MAX], pred[MAX];
    int visited[MAX], count, mindistance, nextnode, i, j;

    // Creating cost matrix
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (Graph[i][j] == 0)
                cost[i][j] = INFINITY;
            else
                cost[i][j] = Graph[i][j];

    for (i = 0; i < n; i++) {
        distance[i] = cost[start][i];
        pred[i] = start;
        visited[i] = 0;
    }

    distance[start] = 0;
    visited[start] = 1;
    count = 1;

    while (count < n - 1) {
        mindistance = INFINITY;
        for (i = 0; i < n; i++)
            if (distance[i] < mindistance && !visited[i]) {
                mindistance = distance[i];
                nextnode = i;
            }

        visited[nextnode] = 1;
        for (i = 0; i < n; i++)
            if (!visited[i])
                if (mindistance + cost[nextnode][i] < distance[i]) {
                    distance[i] = mindistance + cost[nextnode][i];
                    pred[i] = nextnode;
                }
        count++;
    }

    // Printing the distance
    for (i = 0; i < n; i++)
        if (i != start) {
            printf("\nDistance from source to %d: %d", i, distance[i]);
        }
}

int main() {
    int Graph[MAX][MAX], i, j, n, u;
    n = 7;

    // Initialize Graph Adjacency Matrix
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
```

```
            Graph[i][j] = 0;

    Graph[0][2] = 1; Graph[0][3] = 2;
    Graph[1][2] = 2; Graph[1][5] = 3;
    Graph[2][0] = 1; Graph[2][1] = 2; Graph[2][3] = 1; Graph[2][4] = 3;
    Graph[3][0] = 2; Graph[3][2] = 1; Graph[3][6] = 1;
    Graph[4][2] = 3; Graph[4][5] = 2;
    Graph[5][1] = 3; Graph[5][4] = 2; Graph[5][6] = 1;
    Graph[6][3] = 1; Graph[6][5] = 1;

    u = 0;
    Dijkstra(Graph, n, u);
    return 0;
}
```

2. Program Output

```
Distance from source to 1: 3
Distance from source to 2: 1
Distance from source to 3: 2
Distance from source to 4: 4
Distance from source to 5: 3
Distance from source to 6: 3
```